

Automated Hardware Complexity Profiling for Edge-Deployed YOLO Models

William Steve Rodriguez Villamizar (wisrovi rodriguez)

AI Leader & Solutions Architect

eCaptureDtech

Badajoz, Extremadura, Spain

wisrovi.rodriguez@gmail.com

Abstract—Deploying object detection models on edge devices demands strict adherence to hardware constraints, specifically concerning latency, Video RAM (VRAM) consumption, and floating-point operations (GFLOPs). We introduce the *ModelComplexityProfiler*, an automated profiling module integrated directly into a distributed MLOps pipeline. This module executes immediately post-training, evaluating the YOLO model’s architectural and empirical computational cost across multiple image resolutions. By marrying theoretical complexity metrics (GFLOPs, parameter count) with empirical measurements (real-world latency and peak VRAM allocation via `ptflops` and `pynvml`), the profiler provides an actionable, deterministic blueprint that guides subsequent pruning and quantization decisions. We evaluate the profiler across YOLOv8 micro, nano, and small architectures, demonstrating how automated, in-situ profiling accelerates the deployment cycle from training servers to edge devices.

Index Terms—Edge AI, Hardware Complexity, YOLO, MLOps, GFLOPs, Latency Profiling

I. INTRODUCTION

The ubiquitous deployment of Computer Vision systems into edge environments—such as autonomous drones, smart surveillance cameras, and mobile devices—has exacerbated the tension between algorithmic accuracy and hardware capabilities. While modern object detectors like YOLO achieve state-of-the-art mean Average Precision (mAP), their computational overhead often violates the strict thermal, battery, and memory envelopes of edge hardware.

Traditionally, optimizing a model for the edge is a distinct, manual phase decoupled from the training process. Engineers iteratively compress the model via pruning or quantization and subsequently profile it on target hardware. This disjointed workflow introduces significant latency in the MLOps lifecycle. To mitigate this, we propose an automated profiling architecture that evaluates hardware complexity *in-situ*, immediately after the model completes its primary training phase.

II. RELATED WORK

Hardware-aware neural architecture search and profiling have gained significant traction. Tools like `ptflops` and NVIDIA’s `nvml` provide standard interfaces for evaluating GFLOPs and memory [1]. However, these metrics are typically analyzed in isolation. Integrating them into an automated, distributed MLOps pipeline—where metrics instantly dictate deployment viability—represents a critical evolution in Edge AI automation.

Supported by eCaptureDtech.

III. PROPOSED ARCHITECTURE / METHODOLOGY

The *ModelComplexityProfiler* is a deterministic module within the broader WPipe post-training framework. It is invoked automatically upon the successful training of a YOLO model.

A. Theoretical Metrics

Using `ptflops`, the module statically analyzes the PyTorch computational graph to extract the total parameter count and GFLOPs. This provides a hardware-agnostic baseline of the model’s capacity.

B. Empirical Profiling

Theoretical metrics often fail to capture memory bandwidth bottlenecks or optimization nuances of the underlying CUDA architecture. Therefore, the module conducts empirical profiling:

- **Latency:** The model infers a standardized batch of synthetic and real validation images, with timings recorded via high-precision timers.
- **VRAM Allocation:** Using `pynvml`, the module tracks the peak GPU memory footprint during the forward pass.

C. Resolution Scaling

Because edge devices dynamically scale input resolutions to conserve power, the profiler iterates through standard resolutions (e.g., 320, 640, 1024), generating a hardware cost matrix that cross-references image size against latency and VRAM.

IV. EXPERIMENTAL SETUP & IMPLEMENTATION DETAILS

The profiler operates within an ephemeral Docker container on an RTX 3060 (12 GB) worker node. We evaluated the module on three architectures: YOLOv8n, YOLOv8s, and YOLOv8m. The pipeline automatically generates a JSON report containing the complexity matrix, which is then parsed by the pipeline’s reporting LLM.

V. RESULTS & DISCUSSION

The automated profiling revealed non-linear scaling in VRAM consumption as image resolutions increased. For instance, while theoretical GFLOPs scaled quadratically with resolution, empirical latency exhibited step-function increases due to cache misses and memory bandwidth saturation on the

GPU. By automatically surfacing these bottlenecks, the pipeline immediately flagged the YOLOv8m model as non-viable for a 2GB VRAM edge target at 640x640 resolution, prompting an automatic downgrade to YOLOv8s.

VI. BROADER IMPACT / ETHICS STATEMENT

Automating hardware profiling contributes to the democratization of Edge AI, allowing researchers to optimize models without requiring physical access to an array of edge devices. Furthermore, optimizing models for lower computational complexity directly reduces the carbon footprint of deployed sensor networks.

VII. DATA & CODE AVAILABILITY STATEMENT

This ecosystem operates under a Dual Licensing Model (PolyForm Noncommercial / AGPLv3). Code available at <https://github.com/wisrovi/>.

VIII. CONCLUSION & FUTURE WORK

The *ModelComplexityProfiler* bridges the gap between server-side training and edge deployment. By automating the extraction of theoretical and empirical hardware metrics, we accelerate the transition from model design to real-world application. Future work will integrate this profiler into a feedback loop that automatically hyperparameter-tunes the network architecture to satisfy a predefined hardware budget.

ACKNOWLEDGMENT

The author would like to thank eCaptureDtech for providing the necessary infrastructure and environment for this research.

REFERENCES

- [1] P. Dollár, M. Singh, and R. Girshick, “Fast and accurate model scaling,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 924–932.